23CSE455 Design Patterns

Case Study Report

Team: A9
Case study: E-Commerce Application using Spring Boot
Application: Shopizer — open-source e-commerce platform
Date: 2026-09-03
Submission repository: 23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
AM.SC.U4CSE23003	Abhijith S	CSE - A
AM.SC.U4CSE23005	Abhinav G Nair	CSE - A
AM.SC.U4CSE23042	Jithin Jyothi	CSE - A
AM.SC.U4CSE23044	Krishnapriya P	CSE - A

1 Application and architecture

Original source:

https://github.com/shopizer-ecommerce/shopizer; tag 3.2.7

baseline commit : 6a4a0a65a3408ee8f62597b51d1b3aac24b77dee

access date : 2026-09-03

Apache License 2.0

Environment:

Java 11

Spring Boot 2.5.12

Apache Maven

Spring Data JPA/Hibernate ; local profile configures MySQL; application port 8080.

Scope:

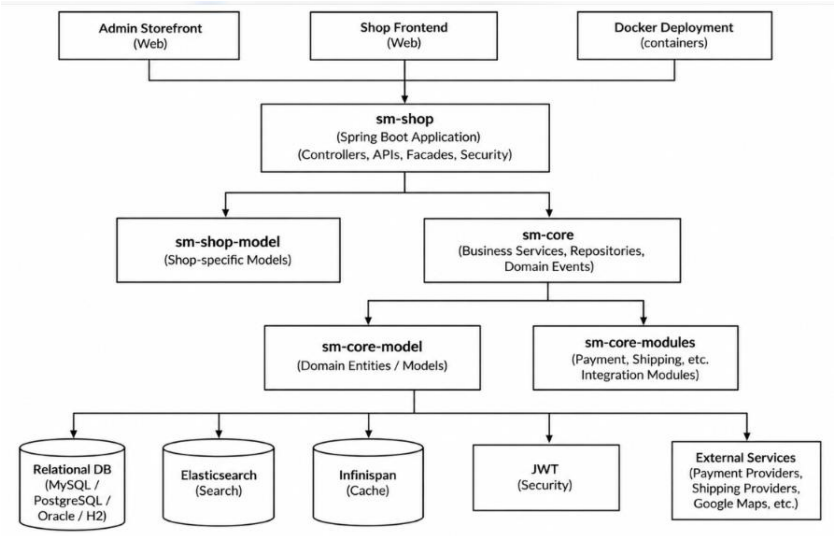
Five Maven modules:

- sm-core-model
- sm-core-modules
- sm-core
- sm-shop-model
- and sm-shop

Focus: payment/shipping integration, product events/search indexing, order facade coordination, security group construction, and CMS image management.

Exclusions: framework internals and candidates not supported by application-level source evidence.

Architecture:



Shopizer follows a modular monolith architecture built using Spring Boot and Maven. The sm-shop module acts as the application entry point, handling REST APIs, controllers, facades, and security. The sm-core module manages business logic and repositories, while sm-core-model contains domain entities, sm-shop-model contains shop-specific models, and sm-core-modules provides integrations such as payment and shipping. The application uses a relational database for persistent storage, Elasticsearch for search, Infinispan for caching, and JWT for security. External services provide payment, shipping, and other integrations where configured.

Request flow: Client to sm-shop to business services in sm-core to database or external integrations. Product events can also trigger listeners to update search indexing.

Run instructions:

./mvnw clean install; cd sm-shop; ../mvnw spring-boot:run; then use http://localhost:8080/swagger-ui.html. README path: README.md (repository root). These are documented instructions.

2 Part 1 Pattern discovery

Method: Inspected application interfaces, concrete implementations, service dispatch points, event publishers/listeners, facade methods, construction helpers, and explicit instance-management code. Confirmed patterns require identifiable source roles and relationships; Spring-only behavior is excluded unless an application-level collaboration is independently evidenced.

2.1 Totals at the baseline

Confirmed pattern types: 5 — Strategy, Observer, Facade, Builder, Singleton.

Confirmed pattern instances: 6 — payment module selection; shipping quote module selection; product events to search indexing; order-processing coordination; security-group construction; CMS product image file manager.

Concrete participant classes: 41 participant occurrences across the six instances (8 + 8 + 12 + 8 + 4 + 1). A participant is a class or interface with a source-evidenced role in the pattern; interfaces are counted, while methods, Spring framework types, enums and unrelated injected dependencies are not.

Unique participant symbols: 41 distinct fully qualified class/interfaces. Methods are not counted as symbols. No class takes part in more than one instance, so unique symbols equal participant occurrences. Classes that share a simple name but sit in different packages (the two CmsImageFileManagerImpl types and the two OrderFacade/OrderFacadeImpl types) are distinguished by fully qualified name.

Rejected and uncertain candidates: 10 register entries (Section 2.3): 1 rejected (order-status Observer); 5 not counted as confirmed application-level patterns (Adapter, Factory, Proxy, Spring default singleton scope, category creation); and 4 elements verified in the source but excluded from participant counts (PayPalRestPayment, canadapost, ProductAttributeEvent and the infinispan CmsImageFileManagerImpl). No candidate is left uncertain.

2.2 Pattern inventory

Pattern type	Instance name	Instances	Participant classes/interfaces	Participant count
Strategy	Payment Module Selection	1	PaymentModule; PaymentServiceImpl; BeanStreamPayment; MoneyOrderPayment; PayPalExpressCheckoutPayment; StripePayment; Stripe3Payment; BraintreePayment	8
Strategy	Shipping Quote Module Selection	1	ShippingQuoteModule; ShippingServiceImpl; USPSShippingQuote; UPSShippingQuote; CustomWeightBasedShippingQuote; StorePickupShippingQuote; PriceByDistanceShippingQuoteRules; CustomShippingQuoteRules	8
Observer	Product Events to Search Indexing	1	PublishProductAspect; ProductImageServiceImpl; ProductEvent; SaveProductEvent; DeleteProductEvent; SaveProductVariantEvent; DeleteProductVariantEvent; SaveProductImageEvent; DeleteProductImageEvent; SaveProductAttributeEvent; DeleteProductAttributeEvent; IndexProductEventListener	12
Facade	Order Processing Coordination	1	OrderFacade; OrderFacadeImpl; OrderService; ShoppingCartService; ShippingQuoteService; CustomerFacade; ShoppingCartFacade; EmailTemplatesUtils	8
Builder	Security Group Construction	1	SecurityGroupsBuilder; Group; Permission; InitializationDatabaseImpl	4
Singleton	CMS Product Image File Manager	1	CmsImageFileManagerImpl (cms.product.local)	1
Total	**5 pattern types**	**6**	**41 participant occurrences = 41 unique fully qualified symbols**	**41**

2.3 One record for each pattern instance

1. Payment Module Selection

Instance name: Payment Module Selection

Pattern type and category: Strategy — GoF behavioural pattern

Purpose and origin: Selects a payment implementation by module key and invokes payment operations through the PaymentModule abstraction; implementations are wired through Shopizer Spring configuration.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Strategy (interface)	com.salesmanager.core.modules.integration.payment.model.PaymentModule	sm-core-modules/src/main/java/com/salesmanager/core/modules/integration/payment/model/PaymentModule.java:16–54
Context / selector	PaymentServiceImpl; paymentModules map	sm-core/src/main/java/com/salesmanager/core/business/services/payments/PaymentServiceImpl.java:52–53, 74–76, 352–378, 433–449, 505, 727–735
Concrete strategies (6)	BeanStreamPayment; MoneyOrderPayment; PayPalExpressCheckoutPayment; StripePayment; Stripe3Payment; BraintreePayment	sm-core/src/main/resources/spring/shopizer-core-modules.xml:45–54, 77–88; classes in sm-core/src/main/java/com/salesmanager/core/business/modules/integration/payment/impl/ (declarations at lines 45, 24, 61, 39, 41, 30 respectively)

Evidence: PaymentServiceImpl retrieves a PaymentModule with paymentModules.get(payment.getModuleName()) (line 352), throws a ServiceException if the module does not exist, and calls authorize, authorizeAndCapture or initTransaction according to the transaction type (lines 374–378). The capture, refund and init flows select the module from the order's payment-module code (lines 433, 505, 727) and call capture (449) and initTransaction (735, 773). The paymentModules map is declared in shopizer-core-modules.xml and binds six provider keys to concrete beans.

Verification: Confirmed. Run-time key-based selection and polymorphic invocation through a shared interface are visible. Participants: 8 (1 interface, 1 context, 6 concrete strategies).

Limitations: PayPalRestPayment implements PaymentModule but is not registered in the paymentModules map and is not referenced elsewhere, so it is excluded. Modules supplied by external starters (ModulesConfiguration injects a List<PaymentModule>) are not in the supplied source. The value-type attribute of the paymentModules map in the XML names com.salesmanager.core.business.modules.integration.payment.model.PaymentModule, which does not match the actual interface package (com.salesmanager.core.modules.integration.payment.model).

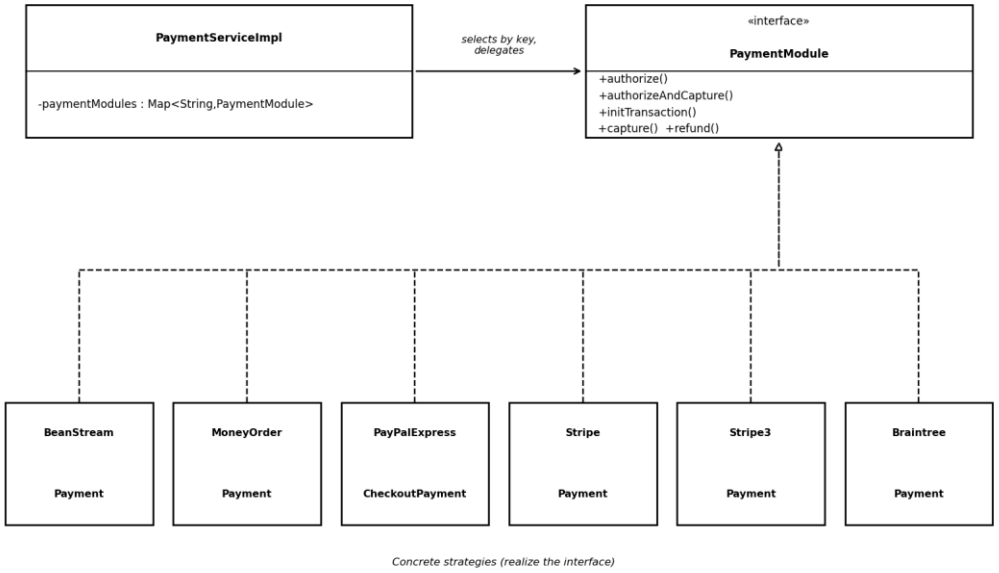


Diagram: *Figure 1. Payment Module Selection: PaymentServiceImpl selects a PaymentModule by key; six concrete modules realize the interface.*

2. Shipping Quote Module Selection

Instance name: Shipping Quote Module Selection

Pattern type and category: Strategy — GoF behavioural pattern

Purpose and origin: Delegates shipping-quote calculation to a selected ShippingQuoteModule implementation.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Strategy (interface)	com.salesmanager.core.modules.integration.shipping.model.ShippingQuoteModule	sm-core-modules/src/main/java/com/salesmanager/core/modules/integration/shipping/model/ShippingQuoteModule.java:19–26
Context / selector	ShippingServiceImpl; shippingModules map	sm-core/src/main/java/com/salesmanager/core/business/services/shipping/ShippingServiceImpl.java:65, 103–105, 459–472, 544–551, 563
Concrete strategies (6)	USPSShippingQuote; UPSShippingQuote; CustomWeightBasedShippingQuote; StorePickupShippingQuote; PriceByDistanceShippingQuoteRules; CustomShippingQuoteRules	sm-core/src/main/resources/spring/shopizer-core-modules.xml:20–35, 58–69; classes in sm-core/src/main/java/com/salesmanager/core/business/modules/integration/shipping/impl/ (declarations at lines 57, 50, 35, 40, 37, 34 respectively)

Evidence: ShippingServiceImpl holds the shippingModules map (lines 103–105). In getShippingQuote it iterates the merchant's configured modules, takes the first active one that is not a ShippingQuotePrePostProcessModule (lines 459–472) and calls getShippingQuotes(...) on it (line 563). A pre-processor may change the current module, in which case the implementation is looked up again in the map (lines 544–551). Other methods use the same map for validation and custom configuration (lines 159–163, 242–246).

Verification: Confirmed as Strategy. It is not classified as Adapter because no target/adaptee/wrapper relationship is demonstrated. Participants: 8 (1 interface, 1 context, 6 concrete strategies).

Limitations: The map also holds the key canadapost, bound to a bean from an external starter whose source is not in the archive; it is excluded. StorePickupShippingQuote also implements ShippingQuotePrePostProcessModule, so the default selection loop skips it and it is reached only through pre-processor re-selection. The XML value-type attribute names a package that does not match the actual interface package.



Diagram: *Figure 2. Shipping Quote Module Selection: ShippingServiceImpl selects a ShippingQuoteModule; six concrete modules realize the interface.*

3. Product Events to Search Indexing

Instance name: Product Events to Search Indexing

Pattern type and category: Observer — GoF behavioural pattern using Spring application events

Purpose and origin: Separates product, variant, image, and attribute changes from the listener that updates search indexing.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Publisher / aspect	PublishProductAspect	sm-core/src/main/java/com/salesmanager/core/business/configuration/events/products/PublishProductAspect.java:34–36, 55–74, 78–97, 101–140
Publisher / service	ProductImageServiceImpl (publishes image events directly)	sm-core/src/main/java/com/salesmanager/core/business/services/catalog/product/image/ProductImageServiceImpl.java:93, 180
Event abstraction	ProductEvent (abstract) and its 8 subclasses: SaveProductEvent; DeleteProductEvent; SaveProductVariantEvent; DeleteProductVariantEvent; SaveProductImageEvent; DeleteProductImageEvent; SaveProductAttributeEvent; DeleteProductAttributeEvent	sm-core/src/main/java/com/salesmanager/core/business/configuration/events/products/ProductEvent.java:7–22; each subclass declared at line 5 or 6 of its file
Observer / listener	IndexProductEventListener	sm-core/src/main/java/com/salesmanager/core/business/configuration/events/products/listeners/IndexProductEventListener.java:39–96, 124–146
Action collaborators (not counted)	SearchService; ProductService	Listener file: lines 42–46, 130, 142

Evidence: PublishProductAspect publishes SaveProductEvent, DeleteProductEvent and the variant, image and attribute event types through Spring's ApplicationEventPublisher after the

matching service methods run. ProductImageServiceImpl publishes the image events directly (lines 93, 180); the comment at line 92 calls this a workaround because the aspect does not fire. IndexProductEventListener implements ApplicationListener<ProductEvent> (line 40), is skipped when search.noindex is true (line 58), dispatches by event subtype (lines 60–90) and calls SearchService.index or deleteDocument (lines 130, 142, 169, 195, 225, 282, 306). Events are delivered on a separate thread (AsynchronousEventsConfiguration.java:17–24).

Verification: Confirmed for product-event/search-index synchronization as a framework-mediated Observer (publish–subscribe) using Spring application events. It does not establish an order-status Observer. Participants: 12 (2 publishers, 9 event classes, 1 listener).

Limitations: The subject and observer interfaces come from Spring; the application supplies the publishers, event types and listener, and there is a single listener. deleteProductImage does not re-index (lines 232–235). ProductAttributeEvent is declared but never published or handled and is excluded.

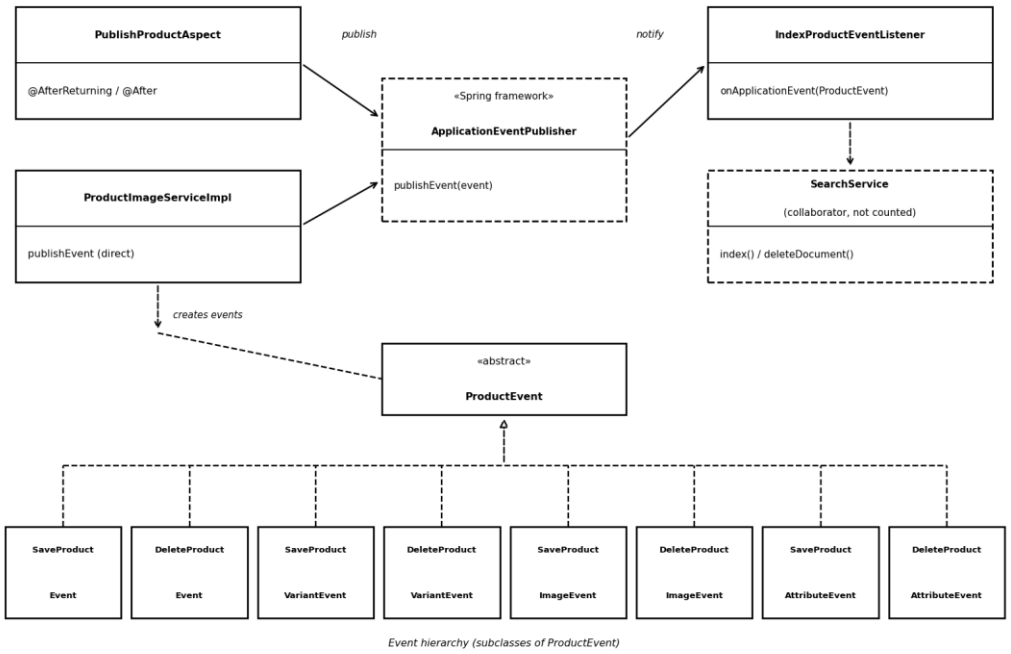


Diagram: *Figure 3. Product events: two publishers create ProductEvent subclasses and publish them through Spring; IndexProductEventListener updates the search index.*

4. Order Processing Coordination

Instance name: Order Processing Coordination

Pattern type and category: Facade — GoF structural pattern

Purpose and origin: Provides a higher-level order workflow API and coordinates underlying services for the shop/API layer.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Facade interface	com.salesmanager.shop.store.controller.order.facade.OrderFacade	sm-shop/src/main/java/com/salesmanager/shop/store/controller/order/facade/OrderFacade.java:31–205

Pattern role	Actual class, interface and function	File and line range
Facade implementation	OrderFacadeImpl	sm-shop/src/main/java/com/salesmanager/shop/store/controller/order/facade/OrderFacadeImpl.java:113–114; fields 118–171
Subsystem collaborators (6)	OrderService; ShoppingCartService; ShippingQuoteService; CustomerFacade; ShoppingCartFacade; EmailTemplatesUtils	OrderFacadeImpl.java: processOrder(v1) 1196–1359 — shoppingCartService 1212; shippingQuoteService 1255; orderService 1273, 1328; customerFacade 1319; shoppingCartFacade 1333; notify → emailTemplatesUtils 1343, 1361–1380

Evidence: OrderFacadeImpl (@Service("orderFacade")) implements OrderFacade. Its processOrder(PersistableOrder, ...) method validates the arguments, maps the request with PersistableOrderApiPopulator, loads the cart (ShoppingCartService, line 1212), obtains the shipping summary (ShippingQuoteService, 1255), calculates totals and persists the order (OrderService, 1273 and 1328), checks the customer (CustomerFacade, 1319), updates the cart (ShoppingCartFacade, 1333) and sends confirmation e-mails through notify (EmailTemplatesUtils, 1365–1377). The older processOrder(ShopOrder, ...) overloads delegate to processOrderModel, which calls OrderService.processOrder (lines 551–553).

Verification: Confirmed as a facade-style subsystem entry point. Core business rules stay in the services. Participants: 8 (facade interface, facade implementation, 6 subsystem collaborators).

Limitations: The class is large (1,648 lines) and also performs mapping and validation. PaymentService, ShippingService and TransactionService are used in other methods (for example captureOrder, line 1418) but outside the order-placement flow counted here, as is the PersistableOrderApiPopulator mapper. A different class named OrderFacadeImpl (com.salesmanager.shop.store.facade.order, implementing the v1 OrderFacade) is not counted.

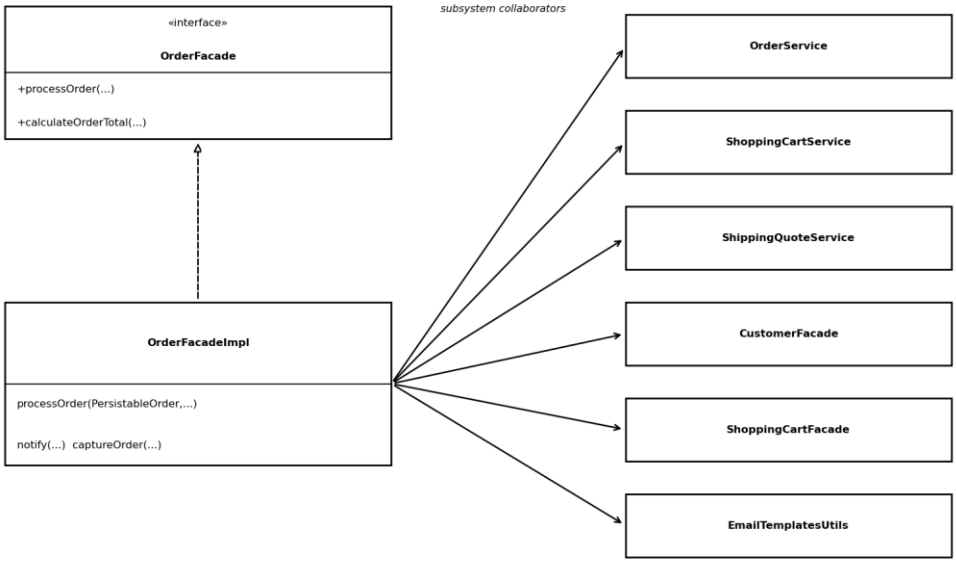


Diagram: *Figure 4. OrderFacadeImpl realizes OrderFacade and coordinates six subsystem collaborators in the order-placement flow.*

5. Security Group Construction

Instance name: Security Group Construction

Pattern type and category: Builder — GoF creational pattern (fluent builder)

Purpose and origin: Builds groups and permissions through chained configuration operations before returning the completed list.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Builder	SecurityGroupsBuilder	sm-core/src/main/java/com/salesmanager/core/business/utils/SecurityGroupsBuilder.java:15–74
Build operations	addGroup(String, GroupType); addPermission(String); addPermission(Permission); build()	Same file: lines 21–30, 32–49, 51–68, 70–72
Product objects	com.salesmanager.core.model.user.Group; com.salesmanager.core.model.user.Permission (GroupType is an enum argument and is not counted)	Group constructed at SecurityGroupsBuilder.java:23 (addGroup; also 36 and 56 in the empty-list fallback); Permission constructed at line 44 (addPermission(String)); types in sm-core-model/src/main/java/com/salesmanager/core/model/user/
Usage site	InitializationDatabaseImpl	sm-core/src/main/java/com/salesmanager/core/business/services/reference/init/InitializationDatabaseImpl.java:50, 172–238

Evidence: addGroup creates a Group, sets its name and type, adds it to the internal list and returns the builder (lines 21–30). addPermission attaches a Permission to the most recently added group and returns the builder (lines 32–68). build() returns the assembled List<Group> (70–72). InitializationDatabaseImpl creates the builder (line 172), chains addGroup/addPermission calls to define eight groups — SUPERADMIN, ADMIN, ADMIN_RETAILER, ADMIN_STORE, ADMIN_CATALOGUE, ADMIN_ORDER, ADMIN_CONTENT and CUSTOMER (173–234) — and persists each group returned by build() through groupService.create (236–238).

Verification: Confirmed as a fluent Builder. It is a small builder and should not be overstated as a complex immutable-object builder. Participants: 4 (builder, 2 constructed types, 1 usage site).

Limitations: There is no abstract builder or director, and only one usage site. If addPermission is called before any group has been added, groups.get(0) fails on the empty list (lines 34, 54) and the following null check is unreachable.

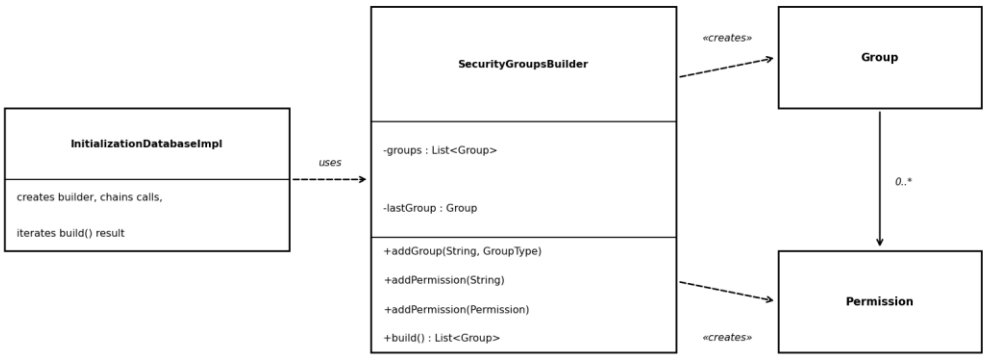


Diagram: *Figure 5. Security Group Construction: InitializationDatabaseImpl chains SecurityGroupsBuilder calls to assemble Group and Permission objects.*

6. CMS Product Image File Manager

Instance name: CMS Product Image File Manager

Pattern type and category: Singleton — GoF creational pattern (explicit, lazy initialization)

Purpose and origin: Provides a static accessor to one lazily created CMS image manager instance.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton class / accessor	CmsImageFileManagerImpl (com.salesmanager.core.business.modules.cms.product.local)	sm-core/src/main/java/com/salesmanager/core/business/modules/cms/product/local/CmsImageFileManagerImpl.java:33–34, 64–72
Instance state	Static fileManager field	Same file: line 43
Private constructor	CmsImageFileManagerImpl()	Same file: lines 74–76
Usage (Spring factory-method)	Bean httpdProductAssetsManager	sm-core/src/main/resources/spring/shopizer-core-cms.xml:118–124; selection through ${config.cms.method}ProductAssetsManager at lines 30–40

Evidence: The static field fileManager (line 43) holds the single instance. getInstance() creates the object only when the field is null and then returns it (lines 66–70). The constructor is private (74–76). A search of the source found no Java call sites for getInstance(); its only use is Spring's factory-method="getInstance" on the httpdProductAssetsManager bean (shopizer-core-cms.xml:118–124).

Verification: Confirmed as an explicit lazy Singleton (implementation evidence); usage evidence is indirect, through Spring XML. The unsynchronised null check makes it not thread-safe as written. Participants: 1.

Limitations: The bean is selected only when config.cms.method=httpd. The default method uses infinispan.CmsImageFileManagerImpl, which has the identical structure (lines 48, 81–94; XML 92–98); it is verified but not counted. The class also implements ProductAssetsManager, which is not a Singleton role.

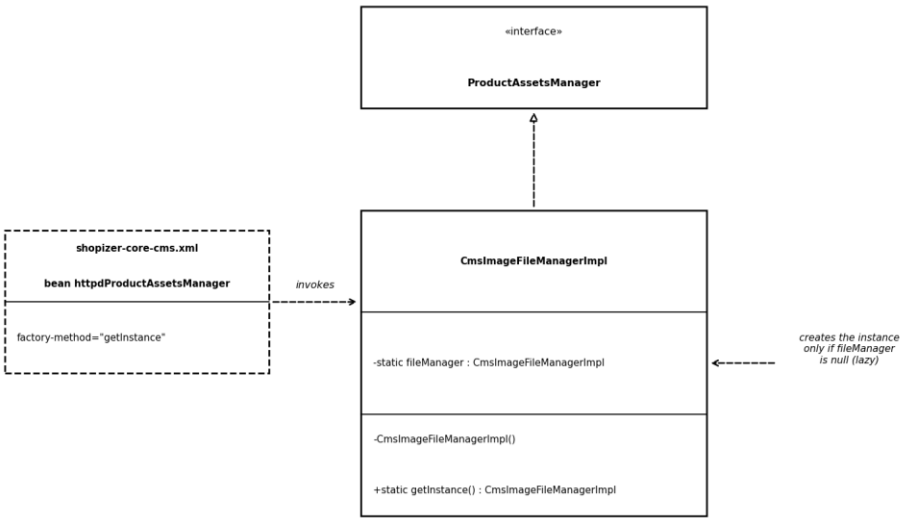


Diagram: *Figure 6. CmsImageFileManagerImpl: static instance field, private constructor and lazy getInstance(), invoked through Spring's factory-method.*

Rejected and uncertain candidates

Candidate	Disposition	Reason / evidence limitation
Adapter — shipping integrations	Not counted as confirmed	Shipping implementations use ShippingQuoteModule directly; no distinct target/adaptee/wrapper collaboration was demonstrated.
Factory — generic object creation	Not counted as confirmed	Spring bean wiring or ordinary object construction alone does not establish a canonical Factory pattern.
Proxy — Spring-generated service proxies	Not counted as application-level GoF instance	Framework AOP/proxy behavior is not counted as a Shopizer-authored Proxy without an explicit application collaboration.
Singleton — default Spring bean scope	Not counted	Framework lifecycle convention; only explicit application-level instance management is counted.
Observer — order-status notification	Rejected for current baseline	No order-status event publisher or listener exists in the supplied source: publishEvent is called only for product events, and the only application listeners are IndexProductEventListener and ServerConfig. The confirmed Observer is product events to search indexing.
Category creation as evolution change	Not a Part 1 pattern candidate	Shopizer already implements category creation/update; this is not pattern evidence by itself.
PayPalRestPayment as a payment Strategy	Excluded from participant counts	Implements PaymentModule but is not registered in the paymentModules map or shopizer-core-modules.xml and has no other references, so PaymentServiceImpl cannot select it.
canadapost shipping module	Excluded from participant counts	Key present in the shippingModules map (shopizer-core-modules.xml:23) but the bean comes from an external starter whose source is not in the archive.
ProductAttributeEvent	Excluded from participant counts	Declared in events/products but never published or handled; the aspect publishes SaveProductAttributeEvent and DeleteProductAttributeEvent instead.
infinispan.CmsImageFileManagerImpl	Verified	Same lazy Singleton structure as the local class (lines 48, 81–94) and used by the default CMS method; excluded so that the inventory keeps one instance per row.

3 Part 2 Evolution and refactoring

CH01 — Razorpay payment integration

Change and acceptance criteria: Requirement: introduce Razorpay as an additional payment option without replacing any existing payment integration and without a dependency upgrade. Acceptance criteria: (1) Razorpay is a PaymentModule implementation registered under its own key in the existing payment-module mechanism; (2) the six existing registered modules (beanstream, paypal-express-checkout, moneyorder, stripe, stripe3,

braintree) remain registered; (3) PaymentServiceImpl selects the module by key with no Razorpay-specific branch; (4) no dependency version is changed. A live Razorpay transaction is not an acceptance criterion and is not claimed.

Before and after: Before: baseline commit 6a4a0a65a3408ee8f62597b51d1b3aac24b77dee. After: commit 9d4f923e83d0658b4935f24ff1bb61692ae720c3.

Changed source: sm-core/src/main/java/com/salesmanager/core/business/modules/integration/payment/impl/RazorpayPayment.java (class declaration at line 47, implementing PaymentModule); registration under the key razorpay in sm-core/src/main/resources/spring/shopizer-core-modules.xml (map entry at line 54, bean declaration at line 99). The six original entries remain at lines 48–53.

Implementation and tests: What changed: RazorpayPayment was added as an implementation of the existing PaymentModule abstraction and registered through the existing Spring payment-module mechanism; existing payment infrastructure was reused where applicable; no dependency was upgraded.

Run command: .\mvnw.cmd -pl sm-core -am "-Dtest=RazorpayPaymentTest" "-Dsurefire.failIfNoSpecifiedTests=false" test. Actual test result: RazorpayPaymentTest, 9 tests, 0 failures, 0 errors, 0 skipped (Surefire report). No live Razorpay transaction is evidenced or claimed.

Pattern effects: Payment Module Selection — extended. RazorpayPayment is added to the existing PaymentModule / PaymentServiceImpl collaboration; the selector and the six existing registered providers are preserved. Shipping Quote Module Selection is unaffected.

Refactoring: None. CH01 is a feature addition.

Before/after metrics: Source LOC scope: Java files under src/main/java and src/test/java in sm-core-model, sm-core-modules, sm-core, sm-shop-model and sm-shop, counting code-bearing lines (blank and comment-only lines excluded) with one Python scanner applied to each commit archive. Main source: 72,018 LOC in 1,167 files before; 72,363 LOC in 1,168 files after (+345 LOC, +1 file). Test source: 3,495 LOC in 37 files before; 3,744 LOC in 38 files after (+249 LOC, +1 file). Test results: 9 of 9 RazorpayPaymentTest tests passing.

CH02 — Category-specific mandatory product attributes

Change and acceptance criteria: Requirement: enforce additional product-attribute requirements for Electronics products without imposing them on unrelated categories. Acceptance criteria: (1) an Electronics category exists for the default merchant; (2) an Electronics product without a brand, held in the existing manufacturer field, is rejected; (3) an Electronics product without exactly one warranty_period attribute is rejected; (4) the selected warranty_period value is accepted only as a whole-number month count from 1 to 120; (5) products of other categories are accepted without these attributes; and (6) the checks run through shared validation on the relevant product and attribute write paths.

Before and after:

Before: commit 9d4f923e83d0658b4935f24ff1bb61692ae720c3 (after CH01).

After: commit b896f59d4c770e7df7e13fb297757ab86e223607, “Require attributes for Electronics products,” together with the later CH02 commits e3db423, “Verify seeded Electronics category persistence,” and 1c12b97, “Refactor Electronics warranty attribute matching.”

In the recorded commit history, e3db423 and 1c12b97 were committed after the CH03 commit a555c07; therefore, the CH02 work is not contiguous in the commit history.

Changed source: InitializationDatabaseImpl.createElectronicsCategory seeds the Electronics category and description (sm-core/src/main/java/com/salesmanager/core/business/services/reference/init/InitializationDatabaseImpl.java, lines 441–461). ElectronicsProductValidator (sm-shop/src/main/java/com/salesmanager/shop/validation/ElectronicsProductValidator.java, lines 18–76) scopes validation to that category and checks the manufacturer and the warranty_period attribute. The product write paths are PersistableProductMapper (sm-shop/src/main/java/com/salesmanager/shop/mapper/catalog/product/PersistableProductMapper.java, lines 243–254) and PersistableProductDefinitionMapper (same directory, line 294). The attribute write path is ProductOptionFacadeImpl, lines 310 and 509.

Implementation and tests: The Electronics category is seeded for the default merchant. ElectronicsProductValidator requires the manufacturer and exactly one warranty_period attribute, whose selected value description must be a whole-number month count from 1 to 120. The validator is applied on the product and attribute write paths listed above.

Run commands: The documented focused Maven runs use .\mvnw.cmd with -pl sm-shop -am and -Dtest=ElectronicsProductValidatorTest, and -pl sm-core -am and -Dtest=ProductNextGenTest, respectively. Actual test results: ElectronicsProductValidatorTest — 7 of 7 tests passing; ProductNextGenTest (product/category persistence) — 2 of 2 tests passing. Test evidence is stored under evidence/design-patterns-part2/. No full Shop API Electronics product-creation integration test exists, so end-to-end testing of that path is not claimed.

Pattern effects: No pattern instance is introduced or removed. Product Events to Search Indexing is preserved: the existing publishers, published product event classes and IndexProductEventListener remain. The Electronics validation is not a GoF pattern instance and is recorded as an excluded candidate in Section 3.1.

Refactoring: Genuine refactoring: commit 1c12b97 in ElectronicsProductValidator. Code smell: the warranty_period matching filter was duplicated. Technique: Extract Method, introducing isWarrantyAttribute at line 60 and reusing one filtered list at lines 46–49 instead of repeating the filter. Behaviour preservation: the evidence document records 7 of 7 validator tests passing before and after the refactoring; however, the retained Surefire report for the validator is the after-state report. The before-state result should be treated as documented evidence rather than independently confirmed by that report. Commits b896f59 and e3db423 are feature and verification work, respectively, and are not counted as refactoring.

Before/after metrics: Source LOC scope: Java files under src/main/java and src/test/java in sm-core-model, sm-core-modules, sm-core, sm-shop-model and sm-shop, counting code-bearing lines (blank and comment-only lines excluded) with one Python scanner applied to each commit archive. For CH02 (b896f59), main source increased from 72,363 LOC in 1,168 files to 72,511 LOC in 1,169 files (+148 LOC, +1 file). Test source increased from 3,744 LOC in 38 files to 3,834 LOC in 39 files (+90 LOC, +1 file). From a555c07 (72,650 main LOC / 4,004 test LOC) to the current committed tree 6b55bc1582c232aba59945f8a0a3d01e02be9a7c, which includes 1c12b97 and e3db423, main source increased by 3 LOC to 72,653 and test source increased by 10 LOC to 4,014, with no change in file counts (1,173 main-source files and 41 test-source files). Test results: 7 of 7 validator tests and 2 of 2 product/category persistence tests passing. The broader sm-core test run and its limitations are discussed in Section 3.1, Interpretation.

CH03 — SMS notifications for order-status changes

Change and acceptance criteria: Requirement: notify customers when an order transitions to a relevant fulfilment status. Acceptance criteria: (1) a successful, persisted transition to SHIPPED or DELIVERED publishes an event; (2) an unchanged status and any other status publish no event; (3) an asynchronous listener receives the event and delegates delivery to an SmsProvider abstraction; (4) the default provider reports that no provider is configured. No SMS vendor adapter or credentials were supplied, so real SMS delivery is not an acceptance criterion and is not claimed.

Before and after: Before: commit b896f59d4c770e7df7e13fb297757ab86e223607. At the baseline 6a4a0a65a3408ee8f62597b51d1b3aac24b77dee, publishEvent was called only for product events and the only application listeners were IndexProductEventListener and ServerConfig, so no order-status event existed. After: a555c073e5cb475da0a58feab965469ea46638cb “Notify customers of order status transitions”.

Changed source: OrderFacadeImpl.updateOrderStatus (sm-shop/src/main/java/com/salesmanager/shop/store/controller/order/facade/OrderFacadeImpl.java:1630–1655); OrderStatusChangedEvent (sm-shop/src/main/java/com/salesmanager/shop/store/controller/order/events/OrderStatusChangedEvent.java:10,19); OrderStatusSmsListener (sm-shop/src/main/java/com/salesmanager/shop/store/controller/order/listeners/OrderStatusSmsListener.java:22,36–67); SmsProvider (sm-shop/src/main/java/com/salesmanager/shop/notification/sms/SmsProvider.java:3,5) and UnconfiguredSmsProvider (same directory, lines 5, 8).

Implementation and tests: What changed: updateOrderStatus returns early if the status is unchanged, persists the history through addOrderStatusHistory, and then publishes OrderStatusChangedEvent only for SHIPPED or DELIVERED. OrderStatusSmsListener is

asynchronous; it filters the statuses, selects a phone number and delegates sending to SmsProvider. The only production implementation, UnconfiguredSmsProvider, reports that no provider is configured.

Actual test results: listener tests 6 of 6 and order-status transition tests 2 of 2 passing, 8 of 8 in the documented command. The tests use a mock provider; there is no SMS vendor adapter and no evidence of real message delivery, which is not claimed. Test evidence is stored under evidence/design-patterns-part2/.

Pattern effects: Order Status SMS Notification — newly introduced: Observer, GoF behavioural pattern realised with Spring application events. Five participant occurrences: OrderFacadeImpl (publisher, reused from the Facade instance), OrderStatusChangedEvent, OrderStatusSmsListener, SmsProvider and UnconfiguredSmsProvider; four are new symbols. It is a separate collaboration from Product Events to Search Indexing. Order Processing Coordination — modified: OrderFacadeImpl.updateOrderStatus now publishes the event after successful persistence for the two SMS statuses, while the status-equality check and the Facade role are retained.

Refactoring: None. CH03 is a feature addition.

Before/after metrics: Source LOC scope: Java files under src/main/java and src/test/java in sm-core-model, sm-core-modules, sm-core, sm-shop-model and sm-shop, counting code-bearing lines (blank and comment-only lines excluded) with one Python scanner applied to each commit archive. Main source: 72,511 LOC in 1,169 files before; 72,650 LOC in 1,173 files after (+139 LOC, +4 files). Test source: 3,834 LOC in 39 files before; 4,004 LOC in 41 files after (+170 LOC, +2 files). Test results: 6 of 6 and 2 of 2 passing; no real SMS delivery is claimed.

3.1 Updated pattern counts

Snapshot (commit)	Pattern types	Instances	Participant occurrences	Unique participant symbols	Main / test LOC	Rejected / uncertain observations
Original (6a4a0a6)	5	6	41	41	72,018 / 3,495	—
After CH01 (9d4f923)	5	6	42	42	72,363 / 3,744	—
After CH02 (b896f59)	5	6	42	42	72,511 / 3,834	—
After CH03 (a555c07)	5	7	47	46	72,650 / 4,004	—
Current tree (6b55bc1)	5	7	47	46	72,653 / 4,014	5 excluded; 1 additional Singleton candidate

Counting method: A participant is a source class or interface that performs a stated role in one of the seven collaborations. Classes and their role interfaces are counted separately. A type that performs roles in multiple instances is counted once per instance in participant occurrences and once in the unique-symbol total. Methods, enums, Spring framework types and helper or action collaborators outside the defined collaboration are excluded.

The original snapshot contains 41 participant occurrences and 41 unique fully qualified symbols, consistent with the Part 1 pattern inventory. Razorpay adds one participant occurrence and one unique symbol to the Payment Module Selection Strategy, increasing its participant count from 8 to 9. CH02 adds no new GoF pattern instance. CH03 introduces the Order Status SMS Notification Observer instance, adding five participant occurrences, four of which are new symbols because OrderFacadeImpl is reused as the publisher.

Consequently, the number of pattern instances increases from six to seven, while the number of pattern types remains five.

The participant counts are based on the defined collaborations and must use the same scope as the Part 1 inventory. PaymentServiceImpl and ShippingServiceImpl should not be counted as Facade participants solely because they exist in the application; their inclusion requires evidence that they participate in the specific Order Processing Coordination collaboration being counted.

SmsProvider is counted as a participant because the SMS listener delegates message delivery through this abstraction. PayPalRestPayment is excluded from the Payment Module Selection Strategy because it is not present in the registration map and has no other references.

Rejected and uncertain observations in the current tree include ProductAttributeEvent (declared but not published in the examined product flow), the history-only order-status API (not a status transition), existing e-mail notification (not SMS), Electronics validation (not a GoF pattern instance), and PayPalRestPayment (an unregistered Strategy candidate).

Additional Singleton candidate: One additional candidate lies outside the seven named instances: com.salesmanager.core.business.modules.cms.product.infinispan.CmsImageFileManagerImpl. It exhibits Singleton structure through a static instance, a lazy accessor and a private constructor. It is excluded from the reported totals because it is outside the seven defined pattern instances. If included as an eighth instance, each snapshot would increase by one participant occurrence and one unique symbol.

Instance name	Effect	Reason
Payment Module Selection	Extended	RazorpayPayment and the razorpay map entry are added; the six existing registered providers and the selector remain.
Shipping Quote Module Selection	Preserved	ShippingServiceImpl, the ShippingQuoteModule contract, the six implementations and their registrations are unchanged.
Product Events to Search Indexing	Preserved	The existing publishers, published product event classes and IndexProductEventListener remain; the SMS collaboration uses a separate event and listener.
Order Processing Coordination	Modified	OrderFacadeImpl.updateOrderStatus publishes an event after successful persistence for SHIPPED and DELIVERED; the status-equality check and the Facade role remain.
Security Group Construction	Preserved	createSecurityGroups continues to use SecurityGroupsBuilder to build Group and Permission instances.
CMS Product Image File Manager	Preserved	The static instance, lazy accessor and private constructor of the local implementation are unchanged.
Order Status SMS Notification	Newly introduced	OrderFacadeImpl publishes OrderStatusChangedEvent; OrderStatusSmsListener filters the statuses and delegates through SmsProvider.

Interpretation: Benefits: the payment Strategy accepted a new provider by extension, which is the intended use of the PaymentModule abstraction; the Observer approach allowed order-status notification to be added with a single event publication in the Facade rather than a direct SMS call; the Electronics rules remained confined to a validator applied on the product and attribute write paths. Trade-offs: the Facade is modified and now also acts as an event publisher; the asynchronous listener means delivery failures do not reach the order transition; with the default provider no SMS is sent. The additional lines of source in each change, and the extra test code, do not by themselves show a better design.

Remaining limitations: no live Razorpay transaction and no real SMS delivery were performed or are claimed, and no full shop-API Electronics product-creation integration test exists. The focused tests pass: 26 in total across Razorpay (9), Electronics validation (7),

SMS listener (6), order-status transition (2) and category persistence (2). The comparable full core-reactor run had one ProductTest UnexpectedRollbackException and 13 skipped tests in both snapshots, and the current Surefire report for ProductTest also records one error (testCreateProduct and testInsertImage). The underlying diagnostic was java.lang.UnsupportedOperationException: getSubject is not supported, raised from javax.security.auth.Subject.getSubject() inside Infinispan 9.4.18.Final during CMS cache-manager initialisation, which prevented product-image persistence. The recorded runs used JDK 25, whereas the POM targets Java 11 and Spring Boot 2.5.12; this is therefore a testing-environment limitation. No Java 11 run has been executed, so no Java 11 result is claimed, the full suite is not claimed to pass, and the failure is not evidence of a defect in the Part 2 code.

4 Part 3 Cross-language comparison

Selected instances: Two representative instances were selected from the Shopizer case study: (1) Strategy — Payment Module Selection, and (2) Observer — Order Status SMS Notification. Payment Module Selection was selected because it demonstrates interchangeable payment implementations selected through a shared contract. Order Status SMS Notification was selected because it demonstrates event publication and listener-based notification added during Part 2. Both are implemented as small, standalone examples in Java, Python, JavaScript and C++.

Common specifications: The same intended behaviours were used across the four language implementations. For Strategy, the implementation supports runtime selection of payment methods, Card and UPI strategies, case-insensitive provider selection, validation of requests, rejection of invalid inputs, duplicate-registration/selection handling where applicable, and extension with additional strategies. For Observer, the publisher announces relevant order-status transitions; observers can respond to SHIPPED and DELIVERED, SMS notification is separated from order processing, an audit observer can coexist with the SMS observer, and missing phone numbers or unavailable SMS-provider functionality are handled without requiring real SMS delivery. The implementations are standalone demonstrations and do not integrate with a live payment gateway or SMS vendor.

Inputs, outputs and error cases: Inputs include payment-method/provider identifiers and payment request data for Strategy, and order-status transition/event data for Observer. Outputs are the selected payment operation/result or observable notification/audit/provider behaviour. Invalid or unsupported payment selections and invalid requests are rejected; irrelevant or unchanged order statuses should not trigger the SMS notification path. The verification notes describe shared expected behaviour, but a separate machine-readable common-fixture file was not identified in the supplied evidence.

Implementations: Source files and test files are located under `A9/A9/Part3/` in Final_Implementation.zip. The README files record the development environments as Java 26, Python 3.14.3, Node.js 24.13.0, and GCC 13.3.0 with C++17 (Ubuntu 24.04 under WSL 2). Run each command from the corresponding language folder shown in the table. The

implementations are standalone pattern demonstrations, not production integrations with Shopizer.

Pattern	Language	Source and test files (relative to A9/A9/Part3/)	Test command	README-recorded environment
Strategy — Payment Module Selection	Java	Strategy_Payment/java/PaymentStrategy.java; TestPaymentStrategy.java	javac PaymentStrategy.java TestPaymentStrategy.java; java -ea TestPaymentStrategy	JDK 26
Strategy — Payment Module Selection	Python	Strategy_Payment/python/payment_strategy.py; test_payment_strategy.py	python -m unittest -v test_payment_strategy.py	Python 3.14.3
Strategy — Payment Module Selection	JavaScript	Strategy_Payment/javascript/payment_strategy.js; test_payment_strategy.js	node --test test_payment_strategy.js	Node.js 24.13.0
Strategy — Payment Module Selection	C++	Strategy_Payment/cpp/payment_strategy.cpp; test_payment_strategy.cpp	g++ -std=c++17 -Wall -Wextra -pedantic test_payment_strategy.cpp -o test_payment_strategy; ./test_payment_strategy	GCC 13.3.0; C++17; Ubuntu 24.04/ WSL 2
Observer — Order Status SMS Notification	Java	Observer_Order_SMS/java/OrderStatusObserver.java; TestOrderStatusObserver.java	New-Item -ItemType Directory -Force build | Out-Null; javac -d build OrderStatusObserver.java TestOrderStatusObserver.java; java -ea -cp build A9.Part3.Observer_Order_SMS.java.TestOrderStatusObserver	JDK 26
Observer — Order Status SMS Notification	Python	Observer_Order_SMS/python/order_status_observer.py; test_order_status_observer.py	python -m unittest -v test_order_status_observer.py	Python 3.14.3

Observer — Order Status SMS Notification	JavaScript	Observer_Order_SMS/javascript/order_status_observer.js; test_order_status_observer.js	node --test test_order_status_observer.js	Node.js 24.13.0
Observer — Order Status SMS Notification	C++	Observer_Order_SMS/cpp/order_status_observer.cpp; test_order_status_observer.cpp	g++ -std=c++17 -Wall -Wextra -pedantic test_order_status_observer.cpp -o test_order_status_observer; ./test_order_status_observer	GCC 13.3.0; C++17; Ubuntu 24.04/ WSL 2

Actual results: I independently compiled/executed the eight test suites from Final_Implementation.zip in the current audit environment. All eight suites passed 10/10 tests each (80/80 total). Audit environment: OpenJDK 21.0.12.1, Python 3.13.5, Node.js 22.16.0, and GCC 14.2.0 using C++17. These are the versions available for this audit, not the README-recorded development versions. The README files separately report the original development results; saved terminal output should be retained with the submission.

Pattern	**Java**	**Python**	**JavaScript**	**C++**	**Total**
Strategy — Payment Module Selection	10/10	10/10	10/10	10/10	40/40
Observer — Order Status SMS Notification	10/10	10/10	10/10	10/10	40/40
Total	20/20	20/20	20/20	20/20	80/80

Static comparison — production LOC: Production LOC is recorded as nonblank, non-comment source lines. Test code is counted separately. Python has the lowest combined production LOC in this dataset (231), followed by C++ (268), JavaScript (308), and Java (335). LOC measures implementation size under the recorded counting rule; it does not by itself establish maintainability, performance or design quality.

Language	**Strategy LOC**	**Observer LOC**	**Combined production LOC**	**Strategy test LOC**	**Observer test LOC**	**Combined test LOC**
Python	80	151	231	75	141	216
C++	98	170	268	61	84	145
JavaScript	107	201	308	107	158	265
Java	101	234	335	143	137	280

Static comparison — language structure: Java uses explicit interfaces and class-based abstractions, a Map-based registry, records for event/result values, enums for status values, and garbage-collected object lifecycle management. Python uses abstract base classes,

dictionaries, frozen dataclasses and enums, with a concise multi-paradigm syntax. JavaScript uses ES6 classes, Map-based lookup and runtime errors for unimplemented methods; its runtime is Node.js/V8. C++ uses abstract classes with pure virtual methods, unordered_map-based lookup, scoped enums, and explicit ownership tools such as std::unique_ptr and RAII. The workbook also records defensive snapshots of observer collections during dispatch in all four languages.

Dynamic comparison and complexity: The workbook and README files do not provide measured maximum function cyclomatic complexity or median process runtime. These remain not recorded; they must not be estimated from LOC or test results. Test execution was independently repeated for this document, but the audit run used different runtime/compiler versions from those listed in the README files.

Required measure	Recorded status
Maximum function cyclomatic complexity per implementation	Not recorded in supplied evidence
Median process runtime per implementation	Not recorded in supplied evidence
Exact run command and terminal log path for each suite	Not included in supplied summary; attach actual logs/README commands
Production LOC	Recorded in cross_language_comparison.xlsx; see table above
Test result	Recorded as 10/10 for each suite; total 80/80

Comparison: The core pattern structures remain consistent across languages: Strategy separates selection from provider-specific payment behaviour through a shared abstraction; Observer separates event publication from the listeners that react to order-status changes. Language-specific syntax changes the form of the abstraction and lifecycle handling, but not the intended collaboration. Java's interfaces and explicit types make contracts visible; Python's ABCs and dataclasses reduce boilerplate; JavaScript relies more on runtime checks; C++ exposes more explicit ownership and dispatch mechanics. The implementations are illustrative, standalone equivalents rather than a direct port of Shopizer's Spring components.

Technology discussion: The comparison illustrates the distinction between design-pattern intent and language mechanism. Strategy relies on composition and polymorphic dispatch, while Observer relies on a publisher/listener relationship and decoupled event handling. Java interfaces, Python abstract base classes, JavaScript classes and C++ pure virtual classes can express similar contracts, although their type checking and lifecycle guarantees differ. The workbook's code-size figures show that equivalent pattern behaviour does not require identical source size. Runtime performance conclusions cannot be drawn because median timing data was not supplied.

Shopizer evolution context: The separate Shopizer metrics in the workbook report six confirmed pattern instances at baseline and seven after Part 2, while the number of distinct pattern types remains five. The additional instance is Order Status SMS Notification (Observer); Razorpay extends the existing Payment Module Selection Strategy, and Electronics validation adds no GoF pattern instance. The workbook reports +726 physical production Java lines and +613 physical test Java lines between its baseline and evolved

snapshots. These physical-line metrics are a separate measurement from the nonblank/non-comment LOC used for the four-language implementations.

5 LLM records and human verification

Run and model: ChatGPT 6, as recorded by the user. The exact access tier, individual run timestamps/date-and-time, and displayed model label for each prompt were not supplied. This section records the prompts provided by the user; it does not claim that every intermediate response is independently archived here.

Inputs and outputs: Inputs included the Shopizer repository (https://github.com/shopizer-ecommerce/shopizer.git), tag/version label 3.2.7, supplied Shopizer source archives, Part 1 and Part 2 report drafts/templates, the final implementation ZIP, and the cross_language_comparison.xlsx workbook. Relevant baseline commit recorded in the Part 1 report: 6a4a0a65a3408ee8f62597b51d1b3aac24b77dee. Outputs included repository recommendations, source inspection/report drafts, cross-language code and tests, run instructions, and verification notes. Exact input/output files should be linked to the team's Part3/llm and evidence folders.

Exact prompts supplied for the major analyses

Prompt 1 — Finding a repository

> I am doing a university case study titled "Design Pattern Identification and Evolution in a Spring Boot-Based E-Commerce Application." Find an open-source e-commerce application developed using Spring Boot. It should have accessible source code and features such as product management, cart, orders, payments, inventory, customers, notifications, and authentication. Compare suitable repositories against these requirements and recommend the best one. Support the recommendation with repository links and source evidence.

Prompt 2 — Checking whether Part 2 changes already exist

> Inspect the supplied Shopizer source before I modify it. The repository is https://github.com/shopizer-ecommerce/shopizer.git, using tag/version label 3.2.7. Check whether these requirements are already implemented:
> 1. Add Razorpay as a payment method.
> 2. Require category-specific product attributes, such as a warranty for Electronics.
> 3. Send SMS notifications when an order's status changes.

Prompt 3 — Making the Part 1 report

> I have attached my Part 1 report draft, the required template, and the shopizer zip. Correct the report so that the application description, architecture, design patterns, class names, source paths, tables, and evidence match the actual project code. Follow the provided template and preserve correct content. Do not invent missing details or state a pattern not present. Give the correct document.

Prompt 4 — Making the Part 2 report

Part 1 is complete, now help me with part 2 using this template, findings, and Shopizer zip. The three changes are Razorpay payment integration, mandatory Electronics product-attribute validation, and SMS notification for order-status changes. For each, explain the requirement, existing design, proposed change, affected components, and impact on relevant patterns. Classify each pattern as preserved, modified, extended, replaced, or removed, with a reason supported by code evidence. Continue page numbering after page 8 and do not repeat part 1.

Prompt 5 — Choosing the two patterns for Part 3

Part 3 – Cross-Language Analysis

Select 2 significant patterns discovered in Parts 1–2.

Implement the same problem using:
• Java
• Python
• JavaScript
• C++

Proceeded with the recommended pattern.

Prompt 6 — Starting Python implementations

Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

Prompt 7 — JavaScript implementation

Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.

Prompt 8 — Java implementation

After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.

Prompt 9 — C++ implementation

Now, implement the same payment strategy and order status observer problems in C++. Also provide the commands to run it.

Prompt 10 — Verification of all four languages

Inspect the final implementation zip and audit both strategy payment module selection and observer order status SMS notification across Java, Python, JavaScript and C++. Check all the documents and verify the correctness.

Claims and evaluation

Verification procedure: The final implementation ZIP was inspected for source files, tests, folder structure and README instructions. The Strategy implementation was reviewed for a common payment contract, interchangeable implementations and selection behaviour. The Observer implementation was reviewed for event publication, observer/listener notification, status filtering and SMS-provider delegation. The recorded test results and production LOC were checked against the supplied summary/counting information, and documentation was reviewed for unsupported claims and command inconsistencies.

Claim / area	Evaluation	Evidence and qualification
Strategy pattern structure in four languages	Supported by recorded manual review and test summary	Common abstraction and interchangeable Card/UPI strategies; exact source-level line references should be linked to the implementation ZIP.
Observer pattern structure in four languages	Supported by recorded manual review and test summary	Publisher/listener separation, SHIPPED/DELIVERED filtering, SMS and audit observers, provider abstraction.
Test outcomes	Recorded as passed	Eight suites × 10 tests = 80/80; retain terminal outputs for independent audit.
Production LOC	Recorded in workbook	Counting excludes blank and comment-only lines; source/test LOC are separated.
Real payment/SMS integration	Not established	No live Razorpay transaction or actual SMS delivery is claimed.
Complexity and median runtime	Cannot confirm from supplied evidence	Numeric cyclomatic-complexity and median-process-time measurements were not provided.
Exact run commands / per-run timestamps / access tier	Not fully documented in supplied summary	Insert only from actual README, terminal history and tool display; do not reconstruct from memory.

Human verification conclusion: The independent audit run confirms that all eight standalone test suites compile/execute successfully in the audit environment, with 80/80 tests passing. This supports the tested functional cases, but does not prove integration with Shopizer's live payment infrastructure or an SMS vendor, nor does it establish performance superiority.

Generated code and how it was tested

Generated code used: The code retained for this exercise implements Strategy — Payment Module Selection and Observer — Order Status SMS Notification in Java, Python, JavaScript and C++. The supplied verification notes report that the test suites were run separately for each pattern and language. Strategy tests cover selection, request validation and invalid input cases; Observer tests cover event registration/dispatch, relevant statuses, observer separation, and unavailable SMS-provider or missing-phone-number cases where implemented. The tests use local/mock behaviour and do not send actual SMS messages or perform real payment transactions.

6 Findings, contributions and references

Findings

- The Strategy and Observer patterns were implemented in four languages with the same stated problem behaviour and 10 reported tests per implementation, for 80/80 reported passing tests overall.
- Production LOC differs across languages: Python 231 combined LOC, C++ 268, JavaScript 308 and Java 335. These values reflect the supplied counting method and do not alone measure code quality.
- Java uses explicit interfaces and strongly structured class contracts; Python uses ABCs and dataclasses; JavaScript expresses contracts more dynamically; C++ uses pure virtual classes and explicit resource ownership mechanisms.
- The Observer example separates order-status event publication from SMS and audit reactions. It uses a provider abstraction and does not claim actual SMS delivery.
- The Strategy example demonstrates runtime selection of payment behaviour and extension through additional strategies. It does not demonstrate a live payment-gateway transaction.
- The Part 2 Shopizer changes increased the recorded confirmed pattern instances from six to seven while retaining five distinct pattern types. Razorpay extended the existing Strategy collaboration; the order-status SMS Observer was newly introduced; Electronics validation was not classified as a GoF pattern.
- The supplied evidence does not include numeric cyclomatic-complexity or median process-time results. These are limitations of the current comparison and should not be inferred from LOC or test pass rates.

Contributions

Contribution record: The following team roster is copied from the Part 1 report. The precise task allocation for Part 3 must be recorded according to the work actually completed by each member; the information supplied for this document does not provide verified roll-number-level allocations for every Part 3 activity.

Roll number	Student name	Batch	Contribution
AM.SC.U4CSE23003	Abhijith S	CSE - A	Responsible for the JavaScript implementation of the Strategy and Observer design patterns. Developed both examples, prepared and executed the corresponding test suites, maintained relevant LLM prompt and output records, and contributed JavaScript-specific measurements, implementation verification, and consistency checks to the cross-language comparison and final report.
AM.SC.U4CSE23005	Abhinav G Nair	CSE - A	Responsible for the C++ implementation of the Strategy and Observer design patterns. Developed both examples, prepared and executed the corresponding test suites, documented the implementation structure and compilation instructions, and contributed C++-specific measurements and verification evidence to the cross-language comparison.
AM.SC.U4CSE23042	Jithin Jyothi	CSE - A	Responsible for the Java implementation of the Strategy and Observer design patterns. Developed the payment-strategy and order-status notification examples, prepared and executed the corresponding test suites, documented the implementation structure and execution instructions, and contributed Java-specific measurements and verification evidence to the cross-language comparison.
AM.SC.U4CSE23044	Krishnapriya P	CSE - A	Responsible for the Python implementation of the Strategy and Observer design patterns. Developed both examples, prepared and executed the corresponding test suites, documented the implementation structure and execution instructions, and contributed Python-specific measurements and verification evidence to the cross-language comparison.

References

1. Shopizer source repository, tag/version 3.2.7. https://github.com/shopizer-ecommerce/shopizer.git. Baseline commit recorded in Part 1: 6a4a0a65a3408ee8f62597b51d1b3aac24b77dee.
2. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994. (Strategy and Observer pattern definitions.)
3. Oracle. Java documentation. https://docs.oracle.com/en/java/
4. Python Software Foundation. Python documentation, including abc and dataclasses. https://docs.python.org/3/
5. OpenJS Foundation / Node.js. Node.js documentation. https://nodejs.org/docs/
6. ISO C++ / cppreference. C++ language and library reference. https://en.cppreference.com/w/
7. Supplied project artifact: cross_language_comparison.xlsx (Language Overview, Pattern Breakdown, Feature Comparison Matrix, Shopizer Evolution Metrics).
8. Supplied project artifact: Human Verification notes and final implementation ZIP/test evidence. Link to the exact team repository paths in the final submission.